



Alliance School of Advance Computing

E1CAA310: Full Stack Development
BCA - IV

Laboratory Manual

Prepared by:
Mr. Veera Raghav K

Approved By:

VISION OF THE UNIVERSITY

Alliance University's vision is to be a world-class University that nurtures talent and catalytically transforms the lives of millions through excellence in teaching, research, service and community development. To uphold a commitment to shaping lives through scholarly teaching and learning, and that which contributes to an equitable and holistic transformation of society at large.

MISSION OF THE UNIVERSITY

The mission of the University is to create and sustain a community of lifelong learners in an environment that emphasizes literacy, critical thinking, and humanistic and scientific inquiry. The University provides a dynamic, challenging and ethical environment for pursuing high quality teaching, research, learning and service across all areas of university, where students, faculty and other key constituents can interact, collaborate and partner with the global community for creation and dissemination of knowledge and transform lives of people through innovation and excellence in higher education.

University Strives to

- Pursue excellence in teaching, learning and scholarship.
- Prepare students for leadership through enlightened learning partnerships with faculty mentors and the community.
- Support faculty and other scholars in pursuing world-class research: clinical, theoretical, empirical, experiential and foster creative endeavour.
- Promote and preserve academic freedom, diversity, equality, harmony and justice.
- Develop mastery of disciplines and professions and instil confidence among its key constituents in their application for a future of meaningful pursuits and productive work in the service of humanity.
- Channelise faculty and student talent for professionally related service to the university, the community and society at large.

Core Values

- Uncompromising Academic Integrity.
- Scholarly work and continuous self-improvement.
- Encouraging and building student ability, character and creativity.
- Pride in Self, University and the Community.

- Leadership, Service, Philanthropy, Social Justice, and Entrepreneurship.

ALLIANCE SCHOOL OF ADVANCE COMPUTING

VISION:

To be a technical institution of repute nationally and internationally in higher learning and research and to produce creative solutions to societal needs.

MISSION:

To provide every individual with a conducive environment suitable to achieve career goals, with a strong emphasis on personality development, and to offer resources to the academically inclined, to gain quality education in all spheres of engineering, applied sciences and management, without compromising the quality and code of ethics to each student of the Institution.

DEPARTMENT OF COMPUTER APPLICATION

Vision

To become a global leader in fostering innovative computer applications through cutting-edge education, research, and entrepreneurship that address evolving societal needs.

Mission

M1. Facilitate meaningful connections between our students and leading industry partners, empowering them to secure rewarding career opportunities based on the industry's competency requirement that align with their skills, interests, and aspirations, thereby fostering a thriving professional community.

M2. Conduct pioneering research that advances the frontiers of computer applications, exploring novel technologies and methodologies to address complex societal challenges and drive innovation in the digital era.

M3. Cultivate an entrepreneurial mindset among our students, providing them with the resources, mentorship, and support needed to transform innovative ideas into successful ventures, driving nation's economic growth and societal impact.

M4. Empower graduates to pursue higher education opportunities at esteemed worldwide institutions, nurturing their intellectual curiosity and leadership potential to become future pioneers and thought leaders in the field of computer applications.

PROGRAM DETAILS

Program Educational Objectives

DATA SCIENCE:

PEO 1. Critically evaluate and interpret data using statistical, machine learning and Data Science techniques

PEO 2. Apply creative problem-solving approaches to develop novel solutions that leverage data science techniques for societal and organizational benefit.

PEO 3. Apply critical thinking skills for solving Complex Data Science problems to make informed decisions.

PEO 4. Raise awareness of concerns pertaining to cyber health, safety, law, the environment, and national and international cultural diversity.

AI&ML:

1. Apply creative and innovative thinking to develop novel AI & ML solutions, contributing to advancements in technology and addressing emerging challenges in industry and society.

2. Evaluate the performance of AI & ML models through experimentation and validation.

3. Critically analyzing data, evaluating AI & ML models, and making informed decisions to improve system performance and outcomes.

GAME DEVELOPMENT:

1. Embrace ethical considerations and social responsibility in game development, including diversity, inclusion, and representation in games.

2. Creatively solve problems encountered during the game development process, including game design challenges, technical issues, and project management hurdles.

3. Drive innovation and enrich gameplay experiences, using advancing technologies and trends within the gaming industry, including virtual reality (VR), augmented reality (AR), artificial intelligence (AI), and procedural generation.

PROGRAM OUTCOMES

PO 1: Disciplinary/Interdisciplinary Knowledge: Apply comprehensive interdisciplinary knowledge, practical skills, and entrepreneurial mindset to address diverse challenges and innovate in chosen fields, fostering adaptability and problem-solving capacity beyond to multidisciplinary field

PO 2: Complex Problem Solving: Solve different kinds of problems in familiar and non-familiar contexts and apply the learning to real-life situations.

PO 3: Critical Thinking: Apply critical thinking skills to analyze, evaluate, and synthesize diverse information, arguments, and evidence to formulate coherent arguments, identify logical flaws, and draw valid conclusions supported by evidence.

PO 4: Creativity: Demonstrate versatility by creatively approaching diverse scenarios, tackling complex problems innovatively, employing multiple perspectives, and leveraging imaginative, lateral thinking, interpersonal skills, and emotional intelligence.

PO 5: Communication Skills: Apply Effectively communicate complex information with clarity and sensitivity through attentive listening, analytical reading, precise writing, articulate oral expression, confident sharing of views, construction of logical arguments using appropriate technical language, and respectful language inclusive of diverse groups.

PO 6: Analytical reasoning/ thinking: Developing critical thinking skills to assess evidence reliability, detect logical fallacies, analyze diverse data, draw valid conclusions, and effectively address opposing viewpoints.

PO 7: Research-Related Skills: Demonstrate proficiency in observing, questioning, problematizing, synthesizing, formulating hypotheses, designing methodologies, employing analytical tools, executing experiments ethically, and reporting findings.

PO 8: Coordinating/Collaborating with each other: Collaborate respectfully within diverse teams, to foster cooperation and efficiently contribute towards common goals.

PO 9: Leadership Readiness/qualities: Formulating an inspiring vision, assembling a capable team, and employing effective management to navigate towards organizational goals while motivating and engaging team members with the vision.

PO 10: Learning how to learn skills: Developing lifelong learning skills for personal, professional, and societal adaptation, encompassing self-directed learning, resource identification, organizational and time management, and fostering a proactive attitude towards continuous development.

PO 11: Digital and Technological Skills: Utilizing ICT for diverse learning and work scenarios, including accessing, evaluating, and employing various information sources, along with utilizing appropriate software for data analysis.

PO 12: Multiculture competence and inclusive spirit: Fostering cultural awareness, multicultural competency, inclusive leadership, gender sensitivity, and empathy towards marginalized groups to promote diversity, equity, and collaboration in various contexts.

PO 13: Value Inculcation: Embracing and embodying constitutional, humanistic, and moral values while promoting responsible global citizenship, ethical conduct, sustainability, and integrity in all endeavors, fostering a culture of truth, peace, and inclusivity.

PO 14: Autonomy, Responsibility and Accountability: Embracing and embodying constitutional, humanistic, and moral values while promoting responsible global citizenship, ethical conduct, sustainability, and integrity in all endeavors, fostering a culture of truth, peace, and inclusivity.

PO 15: Environmental Awareness and Action: Addressing environmental degradation, climate change, and pollution through effective waste management, biodiversity conservation, forest and wildlife preservation, and promotion of sustainable development practices.

PO 16: Community Engagement and Service: Demonstrate the capability to participate in community-engaged services/ activities for promoting the wellbeing of society.

PO 17:Empathy: Demonstrate the ability to identify with or understand the perspective, experiences, or points of view of another individual or group, and to identify and understand other people's emotions

PROGRAM SPECIFIC OUTCOMES

DATA SCEINCE:

PSO1:Assess the quality of data and identify potential biases or inconsistencies.

PSO2:Develop novel algorithms or techniques to extract insights from complex data sets.

GENERATIVE AI:

PSO1: Effectively manage software development projects throughout the entire lifecycle, including project planning, scheduling, resource allocation, risk management, and project documentation.

PSO2: Develop innovative software solutions that address complex computational problems, incorporating advanced algorithms and technologies.

GAME DEVELOPMENT:

1. Implement advanced game physics and sound effects to simulate realistic interactions within the game environment.
2. Integrate artificial intelligence algorithms to create intelligent and dynamic non-player characters (NPCs) and enemies.
3. Adhere to ethical and legal standards in the development and distribution of the game project, including respect for intellectual property rights and responsible use of resources.

Objectives of the Lab

This **Full Stack Development** Lab aims to equip students with the essential skills to design, develop, and deploy complete web applications by building responsive front-end interfaces with HTML, CSS, JavaScript, and modern frameworks, while also developing robust back-end services using technologies such as Java, Node.js, Spring Boot, and REST APIs. Students will gain hands-on experience in integrating databases (SQL and NoSQL), implementing authentication and security, debugging and optimizing across the stack, and applying industry best practices with modern tools to deliver scalable, maintainable, and production-ready applications.

Common Goals :

- Introduce participants to the fundamentals of full stack web development.
- Learn to structure and style responsive user interfaces using HTML, CSS, JavaScript, and modern front-end frameworks.
- Implement interactivity and dynamic behavior with JavaScript and libraries/frameworks such as React or Angular.
- Develop robust back-end applications using Node.js, Java, Spring Boot, and RESTful APIs.
- Work with SQL and NoSQL databases for efficient data storage, retrieval, and management.
- Apply authentication, authorization, and security best practices in web applications.
- Practice debugging, troubleshooting, and performance optimization across the stack.
- Learn best practices for UI/UX design, accessibility, and maintainability.
- Integrate front-end and back-end components into complete applications.
- Deploy and host full stack applications on cloud platforms or web servers.

Overview of the Lab Activities

This lab provides a practical introduction to full stack development, covering both front-end and back-end concepts through hands-on coding exercises. Students will learn to build dynamic, responsive, and data-driven web applications by writing clean, modular, and efficient code, working with modern front-end frameworks (such as React, Angular, or Vue.js) alongside back-end technologies like Node.js, Java, Spring Boot, and REST APIs. Activities will reinforce integration with SQL and NoSQL databases, authentication and security practices, and deployment strategies on cloud platforms. By the end of the lab, students will be equipped with the skills to design, develop, debug, and maintain complete full stack applications, applying industry best practices across the entire development lifecycle.

Safety Precautions and Guidelines

A computer lab is a great resource for learning and working, but it's important to follow safety precautions to avoid harming yourself or the equipment. Here are some key guidelines:

Hardware: Treat monitors, keyboards, and mice gently. Report any malfunctioning equipment to your instructor.

Software: Don't install unauthorized software or tamper with computer settings. Use the lab for its designated purposes.

Personal Information: Be cautious about entering personal data on public lab computers.

Logging Out: Always log out of your account before leaving the computer.

Phishing and Viruses: Don't click on suspicious links or open unknown attachments to avoid malware.

Fire Safety: Know the location of fire extinguishers and evacuation routes. Never fight a large fire, evacuate immediately and alert authorities.

Discipline Instructions

Code of Conduct:

- Adhere to all lab rules and instructions provided by the lab supervisor or instructor.
- Maintain professional behavior and respect towards peers, instructors, and lab staff.
- No food, drinks, or unauthorized electronic devices are allowed in the lab.

Use of Equipment:

- Use lab equipment only for its intended purpose and within the scope of your training.
- Do not modify or disassemble equipment without permission from the lab supervisor.
- Return all tools and equipment to their designated places after use.

Cleanliness and Organization:

- Keep your work area clean and organized.
- Dispose of any waste materials in appropriate containers and clean up spills immediately

Attendance Instructions

Punctuality:

- Arrive on time for all scheduled lab sessions. Late arrivals may not be allowed to participate in the lab activities for safety reasons.
- Sign in upon arrival and sign out when leaving the lab.

Attendance Requirements:

- Attend all scheduled lab sessions. If you cannot attend, notify the lab supervisor in advance and provide a valid reason.
- Make up for any missed lab sessions according to the lab supervisor's instructions.

Participation:

- Actively participate in all lab activities and discussions.
- Complete all assigned tasks and experiments within the lab session time frame.

Syllabus

School: ASAC

Course Code: E1CAA310

Credits: 4

Total Hours: 90

Program: BCA

Course Title: Full Stack Development

Course Type: Core

Hours/Week: 6 Hours

Course Overview	This comprehensive course provides a deep dive into the world of full-stack development, equipping students with the skills to build complete web applications from the front end to the back end. Students will learn to design, develop, and deploy robust and scalable web applications using a variety of programming languages, frameworks, and tools.
Course Objectives	1. Understand the key elements of game design and their impact on player experience. 2. Develop a strong foundation in game mechanics, level design, and narrative development. 3. Apply game design principles to create engaging and innovative game concepts. 4. Collaborate effectively with other team members in a game development environment. 5. Critically analyze and evaluate existing games to identify strengths and weaknesses.

Course Outcomes

CO	Description	BT Level
1	Analyze the impact of game mechanics on player engagement and satisfaction.	3
2	Design and implement engaging game levels that challenge and reward players.	3
3	Develop compelling narratives that drive player motivation and immersion.	4

4	Collaborate effectively with other team members to create cohesive and balanced game designs.	4
5	Critically evaluate existing games to identify strengths, weaknesses, and opportunities for improvement.	3

List of Experiments

1.	Create a basic webpage using HTML tags and structure.	HTML
2.	Apply CSS styling for colors, fonts, and layouts.	HTML/CSS
3.	Design a responsive webpage using Flexbox and Grid.	HTML/CSS
4.	Implement JavaScript DOM manipulation (show/hide elements, change styles).	HTML/CSS/JS
5.	Build a form with validation using HTML, CSS, and JavaScript.	HTML/CSS/JS
6.	Add JavaScript events (click, hover, input) to make the page interactive.	HTML/CSS/JS
7.	Create a simple calculator or to-do list using JavaScript.	HTML/CSS/JS
8.	Develop a multi-page website with navigation links.	HTML/CSS/JS
9.	Use a front-end framework/library (Bootstrap for styling, React basics for components).	HTML/CSS/JS/React
10.	Connect a front-end form to a mock JSON data file and display results.	HTML/CSS/JS/React
11.	Build a basic REST API using Node.js/Express.	HTML/CSS/JS/React/NodeJs/ExpressJs
12.	Perform CRUD operations (Create, Read, Update, Delete) with a database (SQL/NoSQL/Mongo Db).	HTML/CSS/JS/React/MongoDb

13.	Implement user authentication (login/signup) with session handling.	HTML/CSS/JS/React/ MongoDB/PostMan
14.	Integrate front-end and back-end into a complete mini application.	HTML/CSS/JS/React/ DB
15.	Deploy the full stack application on a cloud platform (Netlify/Heroku/ AWS, or Azure).	Netlify/Heroku

Text Books:	
1. <i>Full-Stack Web Development with React and Node</i> by David Mark Clements, Packt , 2023.	
2. <i>The Complete Web Developer Course: Build Professional Websites</i> by Rob Percival	
3. <i>The Complete Web Developer Course: Build Professional Websites</i> by Rob Percival	
Reference Books:	
1. Practical Full-Stack Web Development by Chris Northwood , Apress, 2022.	
2. JavaScript: The Definitive Guide by David Flanagan	
3. Head First HTML, CSS, and JavaScript by Eric Freeman	
MOOC:	
1. https://www.coursera.org/learn/fullstack-web-development	
2. https://www.geeksforgeeks.org/blogs/full-stack-developer-roadmap/	
Case Studies:	
1. Netflix Platform Development	
2. Food cafe Web Application	
3. Swiggy Cloud Kitchen Management	

Assessment methods

SLNo	Assessment Type	Description	Weightage in marks	Course Outcomes To be Assessed					Duration / Time line
				CO 1	CO 2	CO 3	CO 4	CO 5	
1	CA	Assignment	10	√	√	√	√	√	5 Days
2	MSE	MSE	10	√	√				90 Min
3	QZ	Quiz	10	√	√	√	√	√	15 Mins
4	BL	Blended Learning(Cou	5	√	√	√	√	√	10 Hrs

		rsersa)							
5	MP	Project	10	√	√	√	√	√	20 Days
6	AT	Attendance	5	√	√	√	√	√	90 Hrs
7	SEE	Semester End- Examinat ion	50			√	√	√	3 Hrs

CO-PO Mapping

Course Outcome	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PO 13	PO 14	PO 15	PO 16	PO 17
CO1	3	3	3	3	3	3	3	3	2	3	1	N A	N A	N A	N A	N A	NA
CO2	3	3	3	3	3	3	3	3	2	3	1	N A	N A	N A	N A	N A	NA
CO3	3	3	3	3	3	3	3	3	2	3	1	N A	N A	N A	N A	N A	NA
CO4	3	3	3	3	3	3	3	3	2	3	1	N A	N A	N A	N A	N A	NA
CO5	3	3	3	3	3	3	3	3	2	3	1	N A	N A	N A	N A	N A	NA

3-Strong Contribution (SC) 2-Moderate Contribution(MC) 1-Weak Contribution(WC)

CO PSO Mapping

Course Outcome	Program Specific Outcome								
	BCA AI/ML			BCA Game Development			BCA AI/ML		
	PSO 1	PSO 2	PSO 1	PSO 1	PSO 1	PSO 3	PSO 1	PSO 2	PSO 1
CO1	1	CO1	1	CO1	1	CO1	1	CO1	1
CO2	1	CO2	1	CO2	1	CO2	1	CO2	1
CO3	1	CO3	1	CO3	1	CO3	1	CO3	1
CO4	1	CO4	1	CO4	1	CO4	1	CO4	1
CO5	1	CO5	1	CO5	1	CO5	1	CO5	1

List of Experiments

1.	Create a basic webpage using HTML tags and structure.	HTML
2.	Apply CSS styling for colors, fonts, and layouts.	HTML/CSS
3.	Design a responsive webpage using Flexbox and Grid.	HTML/CSS
4.	Implement JavaScript DOM manipulation (show/hide elements, change styles).	HTML/CSS/JS
5.	Build a form with validation using HTML, CSS, and JavaScript.	HTML/CSS/JS
6.	Add JavaScript events (click, hover, input) to make the page interactive.	HTML/CSS/JS
7.	Create a simple calculator or to-do list using JavaScript.	HTML/CSS/JS
8.	Develop a multi-page website with navigation links.	HTML/CSS/JS
9.	Use a front-end framework/library (Bootstrap for styling, React basics for components).	HTML/CSS/JS/React
10.	Connect a front-end form to a mock JSON data file and display results.	HTML/CSS/JS/React
11.	Build a basic REST API using Node.js/Express.	HTML/CSS/JS/React/NodeJs/ExpressJs
12.	Perform CRUD operations (Create, Read, Update, Delete) with a database (SQL/NoSQL/Mongo Db).	HTML/CSS/JS/React/MongoDb
13.	Implement user authentication (login/signup) with session handling.	HTML/CSS/JS/React/MongoDb/PostMan
14.	Integrate front-end and back-end into a complete mini application.	HTML/CSS/JS/React/DB
15.	Deploy the full stack application on a cloud platform (Netlify/Heroku/ AWS, or Azure).	Netlify/Heroku

Ex. No.1: Creating a Basic HTML5 Page with Semantic Structure

Date:

Aim: To create a basic HTML5 page using semantic elements.

Requirements:

- Text Editor
- Web Browser

Theory:

Semantic HTML tags improve accessibility and SEO.

Task:

Create a blog page using semantic tags.

Steps:

1. Create semantic_blog.html.
2. Add semantic tags.
3. Save and open in browser.

Output (Code):

```
<!DOCTYPE html>

<html>

<head>

<title>Semantic Blog</title>

</head>

<body>

<header><h1>My Tech Blog</h1></header>

<nav>

<ul>

<li>Home</li>

<li>About</li>

</ul>

</nav>

<main>

<article>

<h2>Semantic HTML</h2>
```

```
<p>Semantic tags describe content clearly.</p>
</article>
</main>
<footer>&copy;2024</footer>
</body>
</html>
```

Conclusion:

Semantic tags provide meaningful structure.

Ex. No.2: Create a Basic Webpage using HTML**Date:**

Aim: To create a basic webpage using HTML tags.

Requirements:

- Text Editor
- Browser

Theory:

HTML defines webpage structure.

Task:

Create webpage with heading, paragraph and image.

Output:

```
<!DOCTYPE html>
<html>
<body>
<h1>Welcome</h1>
<p>This is my first webpage</p>

</body>
</html>
```

Conclusion:

HTML forms the base of webpages.

Ex. No.3: Apply CSS Styling for Colors, Fonts, and Layouts

Date:

Aim: To apply CSS styling to a webpage for colors, fonts, and layout design.

Requirements:

- Text Editor
- Web Browser

Theory:

CSS (Cascading Style Sheets) is used to control the appearance of HTML elements. It improves visual presentation using properties like color, font-family, margin, and padding.

Task:

Create a webpage and apply CSS styles.

Steps:

1. Create index.html.
2. Create style.css.
3. Link CSS file to HTML.
4. Apply styles.
5. Open in browser.

Output (Code):

index.html

```
<!DOCTYPE html>

<html>

<head>

<title>CSS Styling</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<h1>My Styled Page</h1>

<p>This page uses CSS styling.</p>

<div class="box">Styled Box</div>

</body>
```


</html>

style.css

```
body{  
    background-color: lightyellow;  
    font-family: Arial;  
}  
  
h1{  
    color: blue;  
    text-align: center;  
}  
  
p{  
    color: green;  
    font-size: 18px;  
}  
  
.box{  
    width:200px;  
    height:100px;  
    background-color: orange;  
    padding:20px;  
    margin:20px auto;  
}
```

Conclusion:

CSS enhances webpage appearance and layout.

Ex. No.4: Implement JavaScript DOM Manipulation (Show/Hide Elements, Change Styles)

Date:

Aim: To manipulate HTML elements using JavaScript DOM methods such as show/hide and style changes.

Requirements:

- Text Editor
- Web Browser

Theory:

DOM (Document Object Model) allows JavaScript to access and modify HTML elements dynamically. Using DOM methods, we can change content, hide or show elements, and update styles.

Task:

Create a webpage and manipulate elements using JavaScript.

Steps:

1. Create index.html.
2. Add HTML elements.
3. Write JavaScript functions for DOM manipulation.
4. Open in browser.

Output (Code):

```
<!DOCTYPE html>

<html>

<head>

<title>DOM Manipulation</title>

</head>

<body>

<h2 id="text">Hello World</h2>

<button onclick="hideText()">Hide</button>

<button onclick="showText()">Show</button>

<button onclick="changeColor()">Change Color</button>

<script>

function hideText(){

    document.getElementById("text").style.display="none";
```

```
}  
function showText(){  
    document.getElementById("text").style.display="block";  
}  
function changeColor(){  
    document.getElementById("text").style.color="red";  
}  
</script>  
  
</body>  
</html>
```

Conclusion:

DOM manipulation helps create dynamic and interactive webpages.

Ex. No.5: Build a Form with Validation using HTML, CSS, and JavaScript**Date:**

Aim: To build a form with validation using HTML, CSS, and JavaScript.

Requirements:

- Text Editor
- Web Browser

Theory:

Form validation ensures that the user enters correct and complete information before submission.

Task:

Create a registration form and validate user input.

Steps:

1. Create form using HTML.
2. Style form using CSS.
3. Validate input using JavaScript.
4. Run in browser.

Output (Code):

```
<!DOCTYPE html>
```

```

<html>
<head>
<title>Form Validation</title>
<style>
body{font-family:Arial;}
input{margin:5px;}
</style>
</head>
<body>

<h2>Registration Form</h2>
<form onsubmit="return validateForm()">
Name: <input type="text" id="name"><br>
Email: <input type="text" id="email"><br>
<input type="submit" value="Submit">
</form>

<script>
function validateForm(){
let n=document.getElementById("name").value;
let e=document.getElementById("email").value;
if(n=="" || e==""){
alert("All fields are required");
return false;
}
alert("Form Submitted Successfully");
return true;
}
</script>

</body>

```

</html>

Conclusion:

Form validation prevents incorrect data entry.

Ex. No.6: Add JavaScript Events (Click, Hover, Input)

Date:

Aim: To add JavaScript events to make the webpage interactive.

Requirements:

- Text Editor
- Web Browser

Theory:

Events are actions triggered by the user such as clicking, hovering, or typing input.

Task:

Create webpage with event handling.

Steps:

1. Create HTML elements.
2. Attach JavaScript events.
3. Test interactions.

Output (Code):

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<button onclick="clickEvent()">Click Me</button>
```

```
<p id="demo"></p>
```

```
<input type="text" oninput="inputEvent(this.value)">
```

```
<p id="inputText"></p>
```

```
<script>
function clickEvent(){
    document.getElementById("demo").innerHTML="Button Clicked!";
}
function inputEvent(val){
    document.getElementById("inputText").innerHTML=val;
}
</script>

</body>
</html>
```

Conclusion:

JavaScript events improve user interaction.

Ex. No.7: Create a Simple Calculator using JavaScript**Date:**

Aim: To create a simple calculator using JavaScript.

Requirements:

- Text Editor
- Web Browser

Theory:

JavaScript can perform arithmetic operations dynamically.

Task:

Build a calculator with add and subtract functions.

Steps:

1. Create input fields.
2. Write JavaScript functions.
3. Display result.

Output (Code):

```
<!DOCTYPE html>

<html>
```

```
<body>

<input type="number" id="n1">
<input type="number" id="n2"><br><br>
<button onclick="add()">Add</button>
<button onclick="sub()">Subtract</button>
<p id="result"></p>

<script>
function add(){
  let a=Number(n1.value);
  let b=Number(n2.value);
  result.innerHTML="Result: "+(a+b);
}
function sub(){
  let a=Number(n1.value);
  let b=Number(n2.value);
  result.innerHTML="Result: "+(a-b);
}
</script>

</body>
</html>
```

Conclusion:

Calculator performs operations dynamically using JavaScript.

Ex. No.8: Develop a Multi Page Website with Navigation Links**Date:**

Aim: To develop a multi-page website with navigation links.

Requirements:

- Text Editor
- Web Browser

Theory:

Navigation links connect multiple pages of a website.

Task:

Create Home, About, and Contact pages.

Steps:

1. Create multiple HTML files.
2. Link them using anchor tags.
3. Open in browser.

Output (Code):**index.html**

```
<a href="about.html">About</a> |  
<a href="contact.html">Contact</a>  
<h1>Home Page</h1>
```

about.html

```
<a href="index.html">Home</a>  
<h1>About Page</h1>
```

contact.html

```
<a href="index.html">Home</a>  
<h1>Contact Page</h1>
```

Conclusion:

Navigation allows easy movement between pages.

Ex. No.9:**Use a Front-End Framework (Bootstrap)****Date:**

Aim: To use Bootstrap framework for styling webpages.

Requirements:

- Text Editor
- Web Browser

Theory:

Bootstrap provides ready-made components and responsive layouts.

Task:

Design webpage using Bootstrap.

Output (Code):

```
<!DOCTYPE html>

<html>

<head>

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">

</head>

<body>


<div class="container">

<h1 class="text-primary">Bootstrap Page</h1>

<button class="btn btn-success">Submit</button>

</div>


</body>

</html>
```

Conclusion:

Bootstrap simplifies UI design.

Ex. No.10: Use a Front End Framework (Bootstrap for Styling)**Date:**

Aim: To design a webpage using Bootstrap front end framework for styling and layout.

Requirements:

- Text Editor
- Web Browser
- Internet Connection

Theory:

Bootstrap is a front end framework that provides pre-designed CSS classes and responsive components which make web design faster and easier.

Task:

Create a webpage using Bootstrap components and styles.

Steps:

1. Create index.html.
2. Include Bootstrap CDN link.
3. Add Bootstrap classes to elements.
4. Open in browser.

Output (Code):

```
<!DOCTYPE html>

<html>
<head>
<title>Bootstrap Page</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>

<div class="container mt-4">
  <h1 class="text-primary">My Bootstrap Page</h1>
  <p class="lead">This page is styled using Bootstrap.</p>
  <button class="btn btn-success">Submit</button>
</div>

</body>
</html>
```

Conclusion:

Bootstrap simplifies webpage design and improves responsiveness.

Ex. No.11: Connect a Front End Form to a Mock JSON Data File and Display Results

Date:

Aim: To connect a front end form to a mock JSON data file and display the results.

Requirements:

- Text Editor
- Web Browser

Theory:

JSON (JavaScript Object Notation) is a lightweight data format used to store and exchange data between applications. JavaScript can fetch JSON data and display it on a webpage.

Task:

Create a webpage that loads data from a JSON file and displays it.

Steps:

1. Create data.json.
2. Create index.html.
3. Fetch JSON data using JavaScript.
4. Display the results on webpage.

Output (Code):

data.json

```
[  
  {"name":"Ravi","marks":85},  
  {"name":"Sita","marks":92},  
  {"name":"Kiran","marks":78}  
]
```

index.html

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
  
<title>JSON Data</title>  
  
</head>  
  
<body>  
  
  
  
  
  
  
  
  
  
<h2>Student List</h2>  
  
<button onclick="loadData()">Load Data</button>
```

```
<div id="output"></div>
```

```
<script>
```

```
function loadData(){  
  fetch("data.json")  
  .then(response => response.json())  
  .then(data => {  
    let text = "";  
    data.forEach(item => {  
      text += item.name + " - " + item.marks + "<br>";  
    });  
    document.getElementById("output").innerHTML = text;  
  });  
}  
</script>
```

```
</body>
```

```
</html>
```

Conclusion:

JSON data can be easily connected and displayed using JavaScript.

Ex. No.12: Build a Basic REST API using Node.js and Express**Date:**

Aim: To build a basic REST API using Node.js and Express framework.

Requirements:

- Node.js
- VS Code

Theory:

REST API allows communication between client and server using HTTP methods such as GET and POST. Express simplifies server creation.

Task:

Create a REST API with sample routes.

Steps:

1. Install Node.js and Express.
2. Create server.js.
3. Define API routes.
4. Run the server.

Output (Code):

```
const express = require("express");  
const app = express();
```

```
app.get("/", (req, res) => {  
  res.send("API is running");  
});
```

```
app.get("/users", (req, res) => {  
  res.json([  
    {name:"Ravi"},  
    {name:"Sita"},  
    {name:"Kiran"}  
  ]);  
});
```

```
app.listen(3000, () => {  
  console.log("Server running on port 3000");  
});
```

Conclusion:

Node.js and Express help in building REST APIs efficiently.

Ex. No.13: Perform CRUD Operations (Create, Read, Update, Delete)

Date:

Aim: To perform CRUD operations using a backend server.

Requirements:

- Node.js
- Database (MongoDB / SQL)

Theory:

CRUD operations allow creating, reading, updating, and deleting records from a database.

Task:

Implement CRUD operations for sample data.

Steps:

1. Create API routes for CRUD.
2. Connect database.
3. Test each operation.

Output (Code):

```
app.post("/add", (req,res)=>{  
  res.send("Record Added");  
});
```

```
app.get("/view", (req,res)=>{  
  res.send("Records Displayed");  
});
```

```
app.put("/update", (req,res)=>{  
  res.send("Record Updated");  
});
```

```
app.delete("/delete", (req,res)=>{  
  res.send("Record Deleted");  
});
```

Conclusion:

CRUD operations help manage application data.

Ex. No.14: Implement User Authentication (Login/Signup) with Session Handling

Date:

Aim: To implement user authentication using login and signup with session handling.

Requirements:

- Node.js
- Web Browser

Theory:

Authentication verifies user identity and session handling maintains user login state.

Task:

Create login and signup system.

Steps:

1. Create login form.
2. Validate user credentials.
3. Maintain user session.

Output (Code):

```
<!DOCTYPE html>

<html>

<body>

<h2>Login Form</h2>

<form onsubmit="return loginUser()">

Username: <input type="text" id="user"><br><br>

Password: <input type="password" id="pass"><br><br>

<input type="submit" value="Login">

</form>

<script>

function loginUser(){

    if(user.value=="admin" && pass.value=="123"){
```

```
    alert("Login Successful");
  } else {
    alert("Invalid Username or Password");
  }
  return false;
}
</script>
```

```
</body>
```

```
</html>
```

Conclusion:

Authentication improves application security.

Ex. No.15: Integrate Front End and Back End into a Complete Mini Application**Date:**

Aim: To integrate frontend and backend into a complete mini application.

Requirements:

- Front End (HTML, CSS, JS)
- Back End (Node.js, Express)

Theory:

Integration allows data flow between user interface and server.

Task:

Connect frontend with backend API.

Steps:

1. Create frontend page.
2. Create backend API.
3. Fetch data from backend.

Output (Code):**frontend.html**

```
<!DOCTYPE html>
```

```
<html>
```



```
<body>
```

```
<button onclick="getUsers()">Get Users</button>
```

```
<script>
```

```
function getUsers(){  
  fetch("http://localhost:3000/users")  
  .then(res => res.json())  
  .then(data => console.log(data));  
}
```

```
</script>
```

```
</body>
```

```
</html>
```

backend.js

```
const express = require("express");  
const app = express();
```

```
app.get("/users", (req,res)=>{  
  res.json([ {name:"Ravi"}, {name:"Sita"} ]);  
});
```

```
app.listen(3000, ()=>{  
  console.log("Server started");  
});
```

Conclusion:

Frontend and backend integration completes the mini application.
